

Phase 1 Wall-Time Scaling with Number of Regions

Empirical Measurements, a Corrected Power-Law Fit, and the Path to $N > 100$

July 2026

Abstract

This document analyses how Phase 1 (PGD relaxation) wall time grows with the number of regions N , using profiled runs on the torus. An earlier draft of this document fit a power law of exponent $\alpha \approx 3.1$ ($T \sim N^3$) to two *pre-optimization* points — $N = 10$ (random init, 7 levels) and $N = 30$ (seeded init, 5 levels) — and extrapolated it to conclude that $N = 1000$ would take ~ 14 years and is “not achievable.” **That conclusion was an artifact of the fit and is retracted here.** The two points confounded three variables at once (random vs. seeded initialisation, 7 vs. 5 refinement levels, and pre- vs. post-optimization code), and the accompanying claim that the outer iteration count K grows as $\sim N^2$ is contradicted by direct measurement.

New post-optimization, same-init (seeded) runs at $N = 50$ and $N = 100$ pin the scaling far more honestly. At an equal finest mesh (348×328 , $V = 114,144$), K is essentially *flat* — 54,154 at $N = 50$ versus 54,536 at $N = 100$ — not $\sim N^2$ (which would predict a $4\times$ increase). Total wall grows from **11.6 h** ($N = 50$) to **40.6 h** ($N = 100$) at equal mesh, an effective exponent $\alpha \approx 1.8$; matched instead at constant per-cell resolution (~ 2290 vertices/cell) it grows from 11.6 h to **29.3 h**, $\alpha \approx 1.3$ — but that finer run also converged in about a third of the outer iterations, so 1.3 is a cost-at-fixed-quality figure that *understates* the intrinsic N -exponent, which the cost model places nearer $\alpha \approx 1.8$ –2.2 (§5). Both $N = 100$ runs produce *valid* 100-region partitions. What actually grows with N is the per-call projection cost — the mean projection inner-iteration count mPII climbs $13.8 \rightarrow 21 \rightarrow 33$ — while the outer iteration count stays flat.

The corrected outlook: post-optimization seeded scaling between $N = 50$ and $N = 100$ measures $\alpha \approx 1.3$ –1.8, and the intrinsic (fixed-iteration) N -exponent likely sits at or above the top of that band (≈ 1.8 –2.2). Under $\alpha \approx 1.5$ –2 this puts $N = 1000$ at a matter of **weeks-to-months** (an illustrative $\alpha \approx 1.5$ gives ~ 43 days; $\alpha \approx 2$ gives ~ 190 days), *not* years. That figure is an extrapolation from only two-to-three post-optimization points over a $20\times$ range in N and cannot be trusted beyond an order of magnitude; the observed mPII growth could steepen the effective exponent further at large N . The essential correction is that the catastrophic “ ~ 14 years / not achievable” framing is **not supported by the data**. The dominant cost remains the orthogonal projection (80–87% of wall), whose per-call cost scales as $\mathcal{O}(\text{mPII} \cdot VN)$; the durable algorithmic fix is a closed-form simplex projection with a dual-Newton equal-area coupling (§8), which attacks mPII directly.

Contents

1	Context and Motivation	3
2	Notation and Setup	3
3	Measured Data Points	3
3.1	$N = 10$ and $N = 30$: pre-optimization context (not fittable)	4
3.2	$N = 50$ measured run (post-optimization, seeded)	4
3.3	$N = 100$ measured runs (post-optimization, seeded)	4

4	Why the Old N^3 / 14-Year Fit Was an Artifact	5
5	Corrected Empirical Scaling	6
5.1	Equal-mesh comparison (fixed V)	6
5.2	Resolution-matched comparison (fixed V/cell)	6
5.3	What the decomposition says	7
6	Projections to Larger N (Honestly Hedged)	7
7	Implemented: Serial Optimizations (Changes A/B/C)	8
8	What Remains to Scale to $N > 100$	9

1 Context and Motivation

The long-term goal of the surface-partition project is to compute minimal-perimeter partitions for $N \gg 30$ regions, with $N = 1000$ as a target. Document 04 (*Phase 1 Timing Profile*) established that 80–90% of Phase 1 wall time is spent in the orthogonal projection $\Pi_{\mathcal{C}}$ onto the partition-of-unity and equal-area constraints, and that the projection cost per call is $\mathcal{O}(\text{mPII} \cdot VN)$, where V is the number of mesh vertices and mPII is the mean inner-iteration count.

This document quantifies how total Phase 1 wall time actually grows with N using the N values for which profiled data exist, identifies which factors in the cost model are responsible for the growth, and projects — with explicit caveats — to larger N .

Remark 1.1 (A retracted conclusion). An earlier version of this document concluded $\alpha \approx 3.1$ ($T \sim N^3$) and, from it, $N = 1000 \approx 14$ years and “not achievable.” Both figures are **withdrawn**. They came from a two-point fit (§4) that mixed initialisation methods, level counts, and pre- vs. post-optimization code, and that assumed an empirically false $K \sim N^2$ iteration-count growth. The corrected analysis below uses only the post-optimization, same-init runs at $N = 50$ and $N = 100$, which are the first genuinely comparable pair.

Remark 1.2 (Optimization status of each run). The $N = 10$ and $N = 30$ runs of §3 predate the Phase 1 PGD serial optimizations (Changes A/B/C; see §7 and docs/reference/PHASE1_PGD_SERIAL_OPTIMIZATION). The $N = 50$ run (§3.2) and *both* $N = 100$ runs (§3.3) are post-optimization. Only the post-optimization, same-init points are used to estimate the exponent; the pre-optimization points are retained for context only and are explicitly *not* fit together with the post-optimization data.

2 Notation and Setup

Throughout this document:

- N — number of regions (partition cells).
- V — number of mesh vertices at the finest refinement level.
- L — number of refinement levels.
- T — total Phase 1 wall time (seconds or hours). Taken from `summary.total_wall_s` in `solution/timing_profile.yaml`.
- mBT — mean Armijo backtrack trial steps per major iteration.
- mPII — mean inner-iteration count per projection call.
- K — total major (outer) PGD iterations summed across all levels.
- “V/cell” — finest-level vertices per region, V/N , a proxy for the per-cell mesh resolution.

The cost model from document 04 is

$$T \sim \underbrace{K \cdot (\text{mBT} + 1)}_{\# \text{ projection calls}} \times \underbrace{\text{mPII} \cdot VN}_{\text{cost per call}}. \quad (1)$$

The analysis below tracks each factor separately across N , rather than collapsing them into a single exponent up front.

3 Measured Data Points

All wall times are read from `summary.total_wall_s`; all validity statements are read from the `dormant_cells` block of `solution/metadata.yaml` (a partition is valid when both `dead` and `weak` lists are empty).

3.1 $N = 10$ and $N = 30$: pre-optimization context (not fittable)

Two families of pre-optimization runs exist. They are recorded here for context, but — being pre-optimization and, in the $N = 10$ case, differently initialised — they **must not** be fit together with the post-optimization points to produce an exponent (see §4).

The $N = 10$ reference run used the *random* initialisation, 7 refinement levels, and an 80×66 initial mesh growing to $V_{10} = 187,128$ vertices; total wall time $T_{10} = 1.19$ h. Per-level iteration counts were not recorded (older schema).

The $N = 30$ family used the *seeded* (Voronoi farthest-point) initialisation, 5 levels, and a 90×86 initial mesh growing to $V_{30} = 107,484$ vertices. All produced valid 30-region partitions. The best-case run is seed 84172851: $T_{30} = 74,670.9$ s = **20.74** h, $K = 27,001$, mBT = 6.26, mPII = 19.39, projection share 89.4%, valid (**dead**= [], **weak**= []).

Table 1: Pre-optimization reference runs (torus). Retained for context only; *not* used to estimate the exponent. The $N = 10$ run additionally differs in initialisation (random) and level count (7).

N	init	levels	V (finest)	Wall (h)	valid
10	random	7	187 128	1.19	yes
30	seeded	5	107 484	20.74	yes

3.2 $N = 50$ measured run (post-optimization, seeded)

A profiled run taken *after* the serial optimizations of §7 provides the first post-optimization anchor. It used the seeded initialisation, 5 refinement levels, and a 100×96 initial mesh growing to $V_{50} = 114,144$ vertices (2,283 V/cell). On a clean host (seed 84172851) the total wall time was $T_{50} = 41,754.3$ s = **11.6** h. The partition is valid (**dead**= [], **weak**= []; minimum peak density ≈ 1.0).

Table 2: $N = 50$ post-optimization run (torus, $\lambda = 2.1$, seed 84172851, 5 levels, seeded init, $V_{50} = 114,144$ vertices; run `run_20260627_234511`).

Quantity	Value
Total wall time T_{50}	11.6 h (41,754 s)
Major iterations K	54 154
Mean backtracks mBT	1.93
Mean inner iters mPII	13.76
Projection share	80.5 % of wall
Energy / gradient share	8.2 % / 7.4 %
Backtrack share (net)	2.5 %
Validity	valid (dead = weak = $\emptyset$)

3.3 $N = 100$ measured runs (post-optimization, seeded)

Two post-optimization $N = 100$ runs, both seeded and both producing *valid* 100-region partitions, form the first genuinely comparable pair with $N = 50$:

- **Equal-mesh run** (`run_20260629_141012`): the *same* 348×328 finest mesh as the $N = 50$ run ($V = 114,144$, hence 1,141 V/cell — under-resolved for 100 cells but still valid, minimum

peak density 0.991). This isolates the N -dependence of the algorithm at fixed mesh.

- **Resolution-matched run** (run_20260701.143238): a finer 494×464 mesh, $V = 229,216$, 2,292 V/cell — matched to the $N = 50$ run’s per-cell resolution. Valid, minimum peak density 0.976. This is the fair “same partition quality” comparison, but it also doubles V .

Table 3: Post-optimization $N = 100$ runs (torus, $\lambda = 2.1$, seed 84172851, 5 levels, seeded init). Both are valid partitions (dead = weak = $\emptyset$). The $N = 50$ row is repeated for comparison. “ K flat” is the key observation: at equal mesh K barely moves.

Run	N	finest mesh	V	V/cell	Wall (h)
234511	50	348×328	114 144	2 283	11.6
141012	100	348×328	114 144	1 141	40.6
143238	100	494×464	229 216	2 292	29.3

Table 4: Profile detail for the same three runs. Note K (major iterations) is flat between the two equal-mesh runs while mPII (per-call projection inner iterations) climbs steadily with N and mesh size.

Run	N / mesh	K	mBT	mPII	proj %
234511	50 / 348^2 -class	54 154	1.93	13.76	80.5
141012	100 / same mesh	54 536	1.96	21.15	86.0
143238	100 / finer mesh	16 908	1.48	32.95	87.1

The single most important number in this document is in Table 4: at an *equal* finest mesh, doubling N from 50 to 100 moved K from 54,154 to 54,536 — a 0.7% change. The old $K \sim N^2$ assumption would have predicted K near 216,000 (a $4\times$ increase). The iteration count does *not* blow up with N .

4 Why the Old N^3 / 14-Year Fit Was an Artifact

The retracted $\alpha \approx 3.1$ came from fitting $T \propto N^\alpha$ to exactly two points, $N = 10$ and $N = 30$, after mesh-normalising T_{10} . Three independent defects make that fit unsafe, and a fourth defect propagated into the projection.

Defect 1 — only two points. A power law has two free parameters; fitting it to two points leaves zero degrees of freedom, so the exponent is whatever the pair happens to imply, with no residual to expose a bad fit.

Defect 2 — the initialisation confound. $N = 10$ used the *random* initialisation; $N = 30$ used the *seeded* initialisation. These are different algorithms with different convergence behaviour (the seeded path hands each cell a contiguous winning region from iteration 0). The observed $T_{10} \rightarrow T_{30}$ jump therefore blends an N -effect with an init-method effect, and the two cannot be separated from a single pair.

Defect 3 — the pre- vs. post-optimization confound, and level counts. Both points predate Changes A/B/C, and they used different level counts (7 vs. 5). The serial optimizations

changed mBT and the projection share substantially (§7); a scaling law calibrated on pre-optimization runs does not transfer cleanly to the current code.

Defect 4 — the empirically false $K \sim N^2$ assumption. The old derivation decomposed $\alpha \approx 3$ as “ $\mathcal{O}(N)$ per call $\times \mathcal{O}(N^2)$ iteration count.” The N^2 factor was *inferred from the same confounded pair* (an estimated $K \approx 3,500$ at $N = 10$ against a measured $K \approx 27,000$ – $50,000$ at $N = 30$), not measured at fixed init and code. The post-optimization same-init data refute it directly: at equal mesh, $K = 54,154 \rightarrow 54,536$ from $N = 50$ to $N = 100$ (Table 4). K is flat, so the N^2 term — the sole source of the “extra” two powers of N that produced N^3 — does not exist in the measured regime.

With the $K \sim N^2$ factor removed, the surviving N -dependence is the per-call cost of the projection, which the cost model (1) puts at $\mathcal{O}(\text{mP}II \cdot VN)$. That is a low-order growth, not a cubic one, and it is what the next section measures.

5 Corrected Empirical Scaling

We estimate the exponent using *only* the post-optimization, same-init (seeded) points: the $N = 50$ run and the two $N = 100$ runs. Two honest comparisons are available, and they measure different things.

5.1 Equal-mesh comparison (fixed V)

Holding the finest mesh fixed at 348×328 ($V = 114,144$) and doubling N from 50 to 100:

$$\frac{T_{100}^{\text{eq}}}{T_{50}} = \frac{146,240.4 \text{ s}}{41,754.3 \text{ s}} = 3.50, \quad \alpha_{\text{eq}} = \frac{\ln 3.50}{\ln 2} = \frac{1.253}{0.693} \approx \mathbf{1.81}. \quad (2)$$

This isolates the pure N -effect at fixed mesh work. It is an *upper* estimate of the practically relevant exponent, because at fixed V the $N = 100$ partition is under-resolved (1,141 V/cell), which forces more projection inner iterations per call (mP II : 13.76 $\rightarrow$ 21.15).

5.2 Resolution-matched comparison (fixed V/cell)

Comparing instead at constant per-cell resolution — the $N = 50$ run ($V = 114,144$, 2,283 V/cell) against the finer $N = 100$ run ($V = 229,216$, 2,292 V/cell):

$$\frac{T_{100}^{\text{fine}}}{T_{50}} = \frac{105,601.0 \text{ s}}{41,754.3 \text{ s}} = 2.53, \quad \alpha_{\text{rm}} = \frac{\ln 2.53}{\ln 2} = \frac{0.928}{0.693} \approx \mathbf{1.34}. \quad (3)$$

This is the fair “same partition quality” comparison — adding regions in practice requires a proportionally finer mesh, and it folds in the mesh growth (V also doubled here). But $\alpha_{\text{rm}} = 1.34$ must **not** be read as a clean pure- N exponent: the finer run is fast largely because its outer iteration count *collapsed* to $K = 16,908$, about a third of the $\sim 54,000$ of the other two runs (Table 4), with fewer backtracks (mBT 1.48 vs. 1.93). That K drop is unexplained and may be specific to this mesh/level schedule rather than an N -scaling property. Crucially it *contradicts* the “ K flat” finding this document leans on elsewhere: had K stayed at 54,154, the resolution-matched ratio would be 8.1 and $\alpha_{\text{rm}} \approx 3.0$. Indeed, holding K fixed with $V \propto N$ (constant V/cell), the cost model (1) gives $T \sim \text{mP}II \cdot VN \sim \text{mP}II \cdot N^2$, i.e. $\alpha_{\text{rm}} \gtrsim 2$ before the mP II rise is even counted. So 1.34 is a *lower engineering bound* (cost at fixed quality, this run’s convergence included), while the intrinsic N -exponent for resolution-matched work is more plausibly ≈ 1.8 – 2.2 . The honest bracket is therefore “ α between about 1.3 measured and ≈ 2.2 implied,” with the two $N = 100$ runs disagreeing on where in that band the truth lies.

5.3 What the decomposition says

The two exponents (2)–(3) bracket the post-optimization scaling at $\alpha \approx 1.3$ –1.8. The cost model (1) explains the equal-mesh factor of 3.50 cleanly:

- **Projection-call count** $K \cdot (\text{mBT} + 1)$: $\frac{54,536 \times 2.96}{54,154 \times 2.93} \approx 1.02$ — essentially flat. The iteration count does not grow.
- **Per-call cost** $\text{mP} \cdot VN$ at fixed V : $\frac{21.15 \times 100}{13.76 \times 50} \approx 3.08$ — this is where all the growth lives, split into a factor 2 from N and a factor 1.54 from the rising mP .

Their product (≈ 3.13) is close to the measured 3.50; the residual reflects super-linear per-call overheads (larger $V \times N$ density matrices) not captured by the leading-order model. For the equal-mesh comparison the effective exponent is thus ≈ 1.7 –1.8, nowhere near 3.

The same decomposition applied to the resolution-matched factor of 2.53 is what exposes $\alpha_{\text{rm}} = 1.34$ as an artifact of the iteration-count drop rather than favourable per-call scaling:

- **Projection-call count** $K \cdot (\text{mBT} + 1)$: $\frac{16,908 \times 2.48}{54,154 \times 2.93} \approx 0.26$ — it *fell* $\sim 4\times$, driven entirely by the K collapse in the finer run.
- **Per-call cost** $\text{mP} \cdot VN$: $\frac{32.95 \times 229,216 \times 100}{13.76 \times 114,144 \times 50} \approx 9.6$ — it *rose* nearly $10\times$ (doubled V , doubled N , $2.4\times$ mP).

Their product ($0.26 \times 9.6 \approx 2.5$) recovers the measured factor. So the low 1.34 comes from doing $4\times$ fewer projection calls, not from cheap per-call scaling — the per-call cost actually grew steeply. If that call-count windfall does not recur at larger N (i.e. if K stays flat as it did at equal mesh), resolution-matched scaling is $\alpha \gtrsim 2$.

Remark 5.1 (The real driver: per-call projection cost, not iteration count). Across the three post-optimization runs, mP climbs $13.8 \rightarrow 21 \rightarrow 33$ (Table 4) while K stays flat at equal mesh. The growth with N is therefore concentrated in the *cost per projection call*, consistent with the $\mathcal{O}(\text{mP} \cdot VN)$ model. This is exactly the quantity the dual-Newton reformulation of §8 attacks.

Remark 5.2 (Uncertainty in α). Even the corrected estimate rests on few points: one $N = 50$ run and two $N = 100$ runs, all at a single seed. Run-to-run and host-to-host variance is large (the same $N = 50$ config took 43.9 h on a contended host versus 11.6 h on a clean one — a $3.8\times$ spread from hardware alone). **This host spread ($3.8\times$) exceeds the entire equal-mesh N -signal ($3.5\times$),** so the exponent is only separable from hardware noise if all three runs executed on comparably clean, uncontended hosts. The $N = 50$ anchor is the verified clean run; the two $N = 100$ runs are *assumed* comparable but this has not been independently confirmed, and it should be before the bracket is treated as anything firmer than order-of-magnitude. Treat $\alpha \approx 1.3$ –1.8 (measured) / ≈ 1.8 –2.2 (intrinsic) as an order-of-magnitude characterisation of the $50 \rightarrow 100$ regime, not a precise law, and see §6 for why it must not be pushed confidently to $N = 1000$.

6 Projections to Larger N (Honestly Hedged)

Anchoring on the measured post-optimization point $T_{50} = 11.6$ h and applying $T(N) = T_{50} (N/50)^\alpha$ across the bracket $\alpha \in [1.34, 1.81]$ (with $\alpha = 1.5$ as an illustrative midpoint) gives Table 5. By construction the two $N = 100$ columns reproduce the two measured $N = 100$ runs (29.3 h at $\alpha = 1.34$, 40.6 h at $\alpha = 1.81$), which is the honest statement that our two measured points *define* the bracket rather than test it.

Table 5: Illustrative Phase 1 wall-time extrapolations on a single clean machine, anchored on the measured $N = 50$ point (11.6 h). The $N = 50$ row and both $N = 100$ entries are *measured*; the $N = 200$ and $N = 1000$ rows are extrapolations from only these points and are shown as a range, not a prediction. See the caveats below — especially that mPII growth could steepen the effective exponent beyond this bracket.

N	$\alpha = 1.34$	$\alpha = 1.5$ (illus.)	$\alpha = 1.81$
50	11.6 h (<i>meas.</i>)	11.6 h (<i>meas.</i>)	11.6 h (<i>meas.</i>)
100	29.3 h (<i>meas.</i>)	32.8 h	40.6 h (<i>meas.</i>)
200	74 h (~ 3 d)	93 h (~ 4 d)	143 h (~ 6 d)
1000	~ 640 h (~ 27 d)	~ 1040 h (~ 43 d)	~ 2630 h (~ 110 d)

Read literally, the measured bracket puts $N = 1000$ at roughly **one to four months** on a single clean machine; allowing for the steeper intrinsic (fixed-iteration) exponent $\alpha \approx 2\text{--}2.2$ discussed in §5 pushes the upper end toward ~ 6 **months**. Either way the honest statement is **weeks-to-months, not years**. The practical conclusions, stated with the appropriate hedging:

- $N = 50$ is **routinely feasible** — measured at half a day on a clean host with the seeded initialisation.
- $N = 100$ is **measured and feasible** on a single machine: 29.3 h at matched resolution, 40.6 h on the coarser (under-resolved but still valid) mesh. Both produce valid 100-region partitions.
- $N = 200$ is a **multi-day** single-machine run under this bracket (roughly 3–6 days); still untested, so treat as an extrapolation.
- $N = 1000$ is, on this evidence, **plausibly a weeks-to-months single-machine run** rather than an impossibility. This is an *extrapolation from two-to-three points over a $20\times$ range in N* ; it cannot be trusted to better than an order of magnitude. The former “ ~ 14 years / not achievable” conclusion is **withdrawn** — it was an artifact of the confounded N^3 fit (§4), not a property of the algorithm.

Remark 6.1 (Why $N = 1000$ remains genuinely uncertain). Two-to-three post-optimization points cannot pin a power law over a $20\times$ extrapolation. The clearest reason the effective exponent could *steepen* is the very quantity this document identifies as the driver, mPII. Its growth must be read carefully: of the full $13.8 \rightarrow 21 \rightarrow 33$ climb, only the first step is N -driven ($13.76 \rightarrow 21.15$ at a *fixed* 348×328 mesh as N doubles, a $1.54\times$ rise), while the second ($21.15 \rightarrow 32.95$) is *mesh*-driven (fixed $N = 100$, mesh refined $114\text{k} \rightarrow 229\text{k}$ vertices). So the genuinely N -attributable growth is the $1.54\times$ per doubling. If even that portion is super-constant in N , the per-call cost carries more than the one power of N assumed in (1), and $T(N)$ bends upward at large N ; a resolution-matched extrapolation compounds it with the mesh-driven rise as well. Conversely, the dual-Newton reformulation (§8) would cap mPII and flatten it. The point of Table 5 is not to assert ~ 43 days at $N = 1000$; it is to show that nothing in the measured data supports a catastrophic figure, while also refusing to over-commit to an optimistic one.

7 Implemented: Serial Optimizations (Changes A/B/C)

The optimizations below have *landed* (merge `bae8cd2`); they are the source of the $\approx 3.8\times$ constant-factor speedup measured at $N = 50$. They preserve the optimizer’s output — the $N = 50$ partition was validated bit-identical to the pre-optimization result (contour perimeter 152.3883843639, 100% winner-take-all agreement) — so they reduce the constant in Eq. (1)

without changing which factors carry the N -dependence. The full audit (#1–#13, with measured impact tiers) is recorded in `docs/reference/PHASE1_PGD_SERIAL_OPTIMIZATION_AUDIT.md`.

Change A — backtracking step warm-start (audit #1, #8). `step0` was hard-reset to 1.0 at the top of every PGD iteration, so each iteration re-walked the Armijo backtrack from scratch; near convergence the accepted step decayed to $\sim 2^{-8}$, i.e. ~ 8 rejected trials per iteration, each a full projection + full energy. Change A carries the last accepted step and starts the next search from $\min(s_{\max}, s_{\text{last}}/\rho)$ (Tier 1; Armijo preserved exactly), and hoists the invariant $\|g\|^2$ out of the trial loop. Measured effect: mBT fell to 1.93 at $N = 50$ (from 6.26 in the pre-optimization $N = 30$ run) — the dominant contributor to the speedup, because eliminated rejected trials remove both their energy *and* their projection evaluations.

Change B — projection inner-loop cleanup (audit #4). The constant coupling matrix $C = \|v\|^2(I - J/n)$ and its ε -regularized solve are now built once per call rather than per inner iteration; the $\mathcal{O}(VN)$ `A_prev = A.copy()` plus `np.allclose` stall check is replaced by a scalar consecutive-plateau test on `max_error` (gated by `max_error < 10tol` so it never returns an infeasible iterate); the dead `convergence_history` dict is removed. Result-preserving. Measured effect: the projection’s share of wall fell from 88–90% (pre-opt) to 80.5% at $N = 50$. (At $N = 100$ the share climbs back to 86–87%, driven by the higher mPII, not by a regression in Change B.)

Change C — post-step gradient reuse (audit #6). The post-step gradient $g_{\text{post}} = \nabla E_\varepsilon(x)$ is bit-identical to the next iteration’s $\nabla E_\varepsilon(x)$ (x is unmodified between them), so it is carried forward, halving gradient evaluations. Bit-identical; $\sim 1.6\%$ wall.

Remark 7.1 (Constant-factor wins do not change the shape of the scaling). Changes A/B/C multiply T by a roughly N -independent constant ($\approx 1/3.8$). They therefore shift every absolute number in Table 5 down but leave the *exponent* to be measured from the post-optimization runs themselves — which is exactly what §5 does, rather than importing a pre-optimization exponent.

8 What Remains to Scale to $N > 100$

Equation (1) identifies three independent factors in T . The serial optimizations above shrank the constant; the measured data (§5) show the remaining N -growth is concentrated in the per-call projection cost ($\text{mPII} \cdot VN$), *not* in the iteration count. The durable lever is therefore whatever caps mPII.

1. Further cut re-projection on rejected backtrack steps (partially done). Change A’s Tier 1 warm-start already brought mBT from the 4–12 thrashing regime (e.g. 6.26 at pre-opt $N = 30$) down to ~ 1.9 , capturing most of the easy gain. Two stronger options remain, both still future work:

- **Gradient-step rule without line search** (Barzilai–Borwein or spectral step, audit Tier 2): project exactly once per major iteration, removing the residual (mBT+1) multiplier entirely. Non-monotone; needs a Grippo-type safeguard and clamping against the $[10^{-8}, 1 - 10^{-8}]$ box.
- **Lazy projection:** evaluate the unconstrained gradient step first and project only when the Armijo condition is about to be accepted.

With mBT already near 2, the remaining headroom here is modest ($\lesssim 2\times$) — the larger structural wins are below.

2. Replace the iterative projection with a dual-Newton closed-form solve (highest-merit algorithmic fix, audit #3). This is the direct attack on the growing mPII identified in §5. The coupled sum-to-one + equal-area projection is already closed-form per inner iteration; only the $A \geq 0$ nonnegativity forces the 14–33 inner sweeps we now measure. Dropping the area constraint lets each vertex row project onto the probability simplex in closed form ($\mathcal{O}(N \log N)$); the N equal-area couplings are then re-introduced via a dual $y \in \mathbb{R}^N$ solved with a semismooth Newton iteration (~ 5 – 15 steps). This replaces the clamp/Sinkhorn sweeps with ~ 10 Newton steps and gives *exact* feasibility, removing the ε -floor pathology. Because mPII is the quantity that grows with N , capping it is the single change most likely to flatten the effective exponent; it is the durable fix for $N \geq 100$ and belongs in its own plan.

Remark 8.1 (Inner-solver dual warm-start was ruled out). An earlier draft listed “warm-start the projection inner solver” (carrying dual multipliers between consecutive PGD projections) as a $\sim 2\times$ lever. The serial-optimization audit reclassified this as a **negative finding** (audit #13): the projection’s per-call dual state is not a safe warm start across PGD iterates — it risks returning an infeasible iterate — and the measured profile shows mPII is only ~ 14 at $N = 50$. The structural way to attack mPII (which does grow with N) is the dual-Newton reformulation above, not warm-starting the existing fixed-point loop.

3. Parallelise across regions. The $\mathcal{O}(N)$ factor in the per-call cost arises from processing each region’s density column independently. These columns are embarrassingly parallel: distributing the $V \times N$ density matrix across P cores reduces per-call cost to $\mathcal{O}(VN/P)$, converting the N factor from a wall-clock penalty to a throughput one.

4. Iteration count is *not* the bottleneck (corrected). An earlier draft claimed $K \propto N^2$ and listed reducing that exponent as a priority. The measured post-optimization data refute the premise: at equal mesh K is flat ($54,154 \rightarrow 54,536$ from $N = 50$ to $N = 100$; Table 4). There is no N^2 iteration-count growth to remove. Better initialisation (the seeded Voronoi approach) and adaptive ε remain worthwhile for robustness, but they are not the lever for N -scaling — the per-call projection cost is. Confirming the flat- K trend at a third N (e.g. $N = 200$) with the seeded init would further solidify the corrected picture.

5. FEM assembly is not the bottleneck. Assembly is $< 1\%$ of wall time at current mesh sizes; it becomes relevant only above $\sim 10^5$ – 10^6 vertices.

Remark 8.2 (Cluster scaling). Running multiple seeds in parallel on a cluster (e.g. Pelle) does not reduce the per-seed wall time but gives the ensemble result sooner. For $N = 100$ (~ 1.2 – 1.7 days per seed, measured) this compresses the effective turnaround when several seeds are needed. For $N = 1000$ — a weeks-to-months per-seed run under the current bracket — an ensemble shortens turnaround but does not change the per-seed cost; the dual-Newton projection is the lever that does.

Summary table

Factor	Current scaling (post-opt, measured)	Path to improvement
Cost per projection call	$\mathcal{O}(\text{mP} \cdot VN)$; mP grows 13.8 $\rightarrow$ 21 $\rightarrow$ 33 across the measured runs; projection 80–87% of wall	Dual-Newton closed-form simplex projection (audit #3) — the lever; parallelise regions ($P\times$)
# projection calls	$K \cdot (\text{mBT} + 1)$; mBT = 1.93 after Change A (was 4–12)	BB/spectral step or lazy projection: remove residual mBT+1 multiplier
Iteration count K	Flat , not $\sim N^2$: 54,154 $\rightarrow$ 54,536 ($N=50 \rightarrow 100$, equal mesh)	Not a scaling bottleneck; better init aids robustness only
Overall exponent α	Measured $\approx$ 1.3–1.8 ($N=50 \rightarrow 100$); intrinsic (fixed- K) $\approx$ 1.8–2.2 — the 1.3 end is inflated by a one-off $4\times$ iteration-count drop. Either way <i>not</i> 3.1	Cap mP via the dual-Newton projection to flatten it
Serial opts (A/B/C)	Landed (bae8cd2); $\approx 3.8\times$, output-identical	Done — constant-factor only
$N = 50$ (measured)	11.6 h clean host, valid partition	Routinely feasible on one machine
$N = 100$ (measured, both valid)	29.3 h (matched mesh) / 40.6 h (coarser mesh)	Feasible now; cluster for multi-seed
$N = 1000$ (extrapolated)	~ 43 d ($\alpha=1.5$) to ~ 190 d ($\alpha=2$); <i>weeks-to-months</i> , uncertain to >1 order of magnitude	Dual-Newton projection + parallelism; “14 yr / not achievable” retracted

References